

Artifact Abstract: FeatX

Xutian Li
xtli25@stu.pku.edu.cn
Peking University
Beijing, China

Yanzhen Zou
zouyz@pku.edu.cn
Peking University
Beijing, China

Yifeng Zhu
yifengzhu25@stu.pku.edu.cn
Peking University
Beijing, China

Lu Zhang
zhanglucs@pku.edu.cn
Peking University
Beijing, China

Xianlin Zhao
zhaoxianlin@pku.edu.cn
Peking University
Beijing, China

Bing Xie
xiebing@pku.edu.cn
Peking University
Beijing, China

ACM Reference Format:

Xutian Li, Yifeng Zhu, Xianlin Zhao, Yanzhen Zou, Lu Zhang, and Bing Xie. 2026. Artifact Abstract: FeatX. In *41st IEEE/ACM International Conference on Automated Software Engineering (ASE '26)*, October 12–16, 2026, Munich, Germany. ACM, New York, NY, USA, 2 pages. <https://doi.org/XXXXXXX>. XXXXXXX

Paper title

FeatX: Editing Software by Editing Features for Repository-Level Code Evolution.

Link to the accepted paper.

The accepted-paper preprint is available at <https://arxiv.org/abs/2606.31206>. The artifact is archived at <https://doi.org/10.5281/zenodo.21187017>. Supplementary access points are the public artifact branch <https://github.com/PKU-SoftwareReuse/FeatX/tree/demo-artifact>, the online demo <https://lixutian.github.io/FeatX>, and the demonstration video <https://youtu.be/OZqKZ4li-yM>.

Purpose

FeatX is a feature-oriented interface for LLM-assisted repository evolution. Given an existing Java repository, it extracts a hierarchical feature model, maintains feature-to-code mappings, lets users edit feature descriptions in natural language, and invokes an LLM-backed evolution agent to produce repository-level code patches. The artifact packages the FeatX React frontend, Spring Boot backend, RepoSummary Python feature-extraction module, Docker Compose deployment, a precomputed NBlog case-study seed, and a replay dataset of 38 real-world feature-editing commits from FlappyBird, PlayEdu, and NBlog. Reviewers can exercise the web tool, inspect the seeded feature map and matching source snapshot, validate the dataset schema and counts, and check component-level buildability. The quick checks intentionally avoid rerunning all LLM-backed experiments; full

feature extraction and code evolution require reviewer-provided OpenAI-compatible API credentials.

Badge

We request the **Reusable** and **Available** badges. The artifact deserves the Reusable badge because it includes the complete tool source, Dockerfiles, Docker Compose configuration, prebuilt container images when provided through the archive, curated seed data, dataset documentation, component-level checks, and explicit documentation of which claims require external LLM credentials. It is documented, internally consistent, and exercisable through smoke tests for the frontend, backend, MySQL seed, RepoSummary environment, and dataset. The artifact deserves the Available badge because it is archived on Zenodo with DOI <https://doi.org/10.5281/zenodo.21187017> and is distributed under the MIT License.

Technology skills assumed by the reviewer evaluating the artifact and hardware requirements.

The recommended evaluation path assumes basic Linux shell usage, Docker Engine with Docker Compose v2, curl for HTTP checks, and jq for JSON dataset inspection. The packaged Docker path targets Linux x86_64/amd64, requires no GPU or non-commodity peripherals, and is intended for a CPU-only machine with 16 GiB RAM and 20 GiB free disk space. A host-installed MySQL server, Java runtime, Node.js runtime, or Python virtual environment is not required for the Docker path. Manual deployment is documented but not recommended for artifact evaluation; it requires Java 17, Node.js 20, Python 3.10, MySQL 8, and component-specific configuration.

Provenance

The artifact can be obtained from the Zenodo archive <https://doi.org/10.5281/zenodo.21187017>. The package contains README.md, REQUIREMENTS.txt, STATUS.txt, LICENSE.txt, AE_Abstract.pdf, Frontend/, Backend/, RepoSummary/, docker-compose.yml, docker/, datasets/commits/dataset.json, datasets/mysql/featx_seed.sql, and datasets/repos/12. The prebuilt image archive FeatX_ASE26_docker_images_20260703.tar.gz contains featx-backend:ase26, featx-frontend:ase26, and featx-mysql:ase26. The NBlog seed contains only the FeatX application tables used by the case study

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ASE '26, Munich, Germany

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
<https://doi.org/XXXXXXX.XXXXXXX>

and excludes API credentials, MySQL users, logs, and unrelated operational data.

Instructions

Reviewers should use the Docker Compose path from the artifact root. First create the local configuration file:

```
cp .env.example .env
```

If the prebuilt image archive is supplied, load it and start the stack:

```
docker load -i \
  /path/to/FeatX_ASE26_docker_images_20260703.tar.gz
docker compose up -d
```

Otherwise build from source with `docker compose build` before `docker compose up -d`. Then run:

```
docker compose ps
curl -i http://localhost:8080/connect/test
curl -i http://localhost:3000/
```

```
curl -i http://localhost:3000/api/connect/test
```

The expected result is that MySQL, backend, and frontend are running; the backend endpoint returns HTTP 200; the frontend returns the FeatX HTML page; and the frontend proxy to `/api/connect/test` returns HTTP 200. If ports 8080 or 3000 are occupied, set `BACKEND_PORT` and `FRONTEND_PORT` in `.env`. Dataset validation is:

```
jq '[.dataset[] | .issues | length] | add' \
  datasets/commits/dataset.json
```

which should return 38. The seeded NBlog UI example opens `http://localhost:3000/`, selects the NBlog project, expands Blog content management and retrieval, and inspects the CodeMap and Diff panels for a packaged feature. Startup, seeded-data inspection, dataset checks, and this non-LLM example do not require API keys. Full LLM-backed feature extraction and code evolution require reviewer-only credentials in `.env`, including `LLM_API_KEY` and `OPENAI_API_KEY`.